BroasterGo: mejoras al proyecto

Explicación de lo que se agregó, por qué y cómo usarlo.

Rama: `mejoras/reglas-negocio-y-pruebas`

9 de noviembre de 2026

Resumen

- Se corrigió un error de cálculo: el IGV se cobraba también sobre el descuento.
- Se programaron las reglas del README que faltaban: delivery por zona, pedido mínimo, cupones, forma de pago y alerta de stock (R4 a R8).
- El sistema POO ahora valida el correo del cliente, atiende varios pedidos seguidos y termina con el cierre de caja.
- La ventana (Tkinter) permite quitar artículos, muestra el subtotal mientras se arma el pedido y ya no acepta descuentos fuera de rango.
- Se agregaron 24 pruebas automáticas.

Lista de cambios

#	Mejora	Archivo(s)	Para qué sirve
1	Descuento antes del IGV	calculos.py	Cobrar el monto correcto al cliente.
2	Módulo compartido	calculos.py (nuevo)	La carta y las cuentas están en un solo lugar.
3	Reglas R4 a R8	broaster_go_pedidos.py	Delivery, pedido mínimo, cupones, pago y alerta de stock.
4	Validar cliente	broaster_go_pedidos.py	Evitar correos mal escritos o datos vacíos.
5	Varios pedidos + cierre de caja	broaster_go_pedidos.py	El stock se mantiene entre pedidos y se ve el resumen del día.
6	Ventana mejorada	broaster_go_gui.py	Quitar artículos, subtotal en vivo, validar descuento.
7	Orden del repositorio	.gitignore (nuevo), los 3 programas	No subir archivos basura y poder importar el código.
8	Pruebas automáticas	test_pedidos.py, test_calculos.py (nuevos)	Comprobar las reglas con un solo comando.

Cómo ejecutar todo

No hace falta instalar nada: todo usa la biblioteca estándar de Python 3.10 o superior.

```
python broaster_go_pedidos.py   # sistema POO: stock, delivery, cupones, cierre de caja
python broaster_go_gui.py       # ventana de mostrador
python broaster_go.py           # mostrador en consola con IGV
python -m unittest -v           # corre las 24 pruebas automáticas
```

1. Corrección: el descuento va antes del IGV

En `broaster_go.py` y `broaster_go_gui.py` el total se calculaba así: `subtotal + IGV - descuento`. El problema es que el IGV se calculaba sobre el subtotal *completo*, incluida la parte que el cliente no paga por el cupón. En Perú el IGV se calcula sobre el valor de venta ya descontado.

Ejemplo: subtotal S/ 100 con cupón de 10 %	**Antes**	**Ahora**
Descuento	S/ 10.00	S/ 10.00
Base para el IGV	S/ 100.00 (incorrecto)	S/ 90.00
IGV 18 %	S/ 18.00	S/ 16.20
Total	**S/ 108.00**	**S/ 106.20**

El cálculo ahora vive en una sola función, con el orden correcto:

```
def calcular_totales(subtotal, porcentaje_descuento):
    porcentaje = limitar_porcentaje(porcentaje_descuento)   # siempre entre 0 y 100
    descuento = round(subtotal * porcentaje / 100, 2)
    base = subtotal - descuento          # 1) primero se resta el descuento
    impuesto = round(base * IGV, 2)      # 2) el IGV se calcula sobre la base
    total = round(base + impuesto, 2)
```

Nota: en la ventana también había otro error. El campo “Descuento (%)” aceptaba cualquier número. Con 150 % el total salía **negativo** (por ejemplo, -S/ 23.68). Ahora la ventana muestra “El descuento debe ser un numero entre 0 y 100” y la consola vuelve a preguntar.

2. Nuevo módulo compartido: calculos.py

Antes, la carta de productos y el `IGV = 0.18` estaban **copiados** en la consola y en la ventana. Si cambiaba un precio había que acordarse de cambiarlo en dos archivos. Ahora ambos programas importan lo mismo desde un solo archivo (principio *“no te repitas”*):

```
from calculos import CARTA, IGV, calcular_totales, soles

soles(12.5)   # -> 'S/ 12.50'   (función pequeña para dar formato de dinero)
```

`calculos.py` contiene: `CARTA`, `IGV`, `limitar_porcentaje()`, `calcular_totales()` y `soles()`.

3. Reglas de negocio en el sistema POO

El README (sección 6.1) describía 8 reglas, pero `broaster_go_pedidos.py` solo tenía las tres primeras. Ahora están todas. Sus valores están como **constantes al inicio del archivo**: para cambiar una tarifa o crear un cupón nuevo basta con editar una línea.

```
STOCK_MINIMO = 3                                    # R8
PEDIDO_MINIMO_DELIVERY = 25.0                       # R4
TARIFAS_DELIVERY = {"A": 3.0, "B": 5.0, "C": 8.0}   # R5
CUPONES = {"BROASTER10": 10, "FAMILIA15": 15}       # R6
FORMAS_PAGO = ["Efectivo", "Yape", "Plin", "Tarjeta"]  # R7
```

Regla	Qué hace el programa	Si no se cumple
R1 Cantidad	La cantidad debe ser mayor que 0.	Ese producto no entra; se anota en observaciones.
R2 Precio	El precio del producto debe ser mayor que 0.	Ese producto no entra; se anota en observaciones.
R3 Stock	No se vende más de lo que hay.	Ese producto no entra; se indica el stock disponible.
R4 Pedido mínimo (nuevo)	Para delivery, los productos deben sumar al menos S/ 25.00.	Pedido **rechazado** y el stock se devuelve.
R5 Delivery por zona (nuevo)	Suma S/ 3, S/ 5 o S/ 8 según la zona A, B o C.	Zona inexistente: pedido rechazado.
R6 Cupones (nuevo)	BROASTER10 = 10 %, FAMILIA15 = 15 %. No importan mayúsculas.	Cupón inexistente: no hay descuento y se avisa.
R7 Forma de pago (nuevo)	Efectivo, Yape, Plin o Tarjeta.	Pedido rechazado.
R8 Alerta de stock (nuevo)	Si después de vender quedan 3 o menos, avisa para reponer.	(Es solo un aviso, la venta sigue.)

Fórmula del total

```
total = subtotal de productos - descuento del cupón + costo de delivery
```

Ejemplo: 4 combos (S/ 72) + 2 papas (S/ 12) = S/ 84.00 → cupón FAMILIA15 (-S/ 12.60) → zona B (+S/ 5.00) = **S/ 76.40**.

¿Qué significa “devolver el stock”?

El programa va separando el stock producto por producto. Si al final el pedido se rechaza (por ejemplo, delivery de S/ 17 cuando el mínimo es S/ 25), devuelve lo que había separado. Así el inventario no baja por un pedido que nunca se vendió:

```
if not pedido_valido:
    for producto, cantidad, _ in items_confirmados:
        producto.stock += cantidad      # se devuelve lo separado
```

4. Validación de los datos del cliente

Nuevas funciones `validar_correo()` y `validar_cliente()`. Si algo está mal, el programa explica qué falta y vuelve a pedir los datos. El código y el nombre no pueden quedar vacíos.

Correo escrito	**¿Se acepta?**	**Motivo**
ana@gmail.com	Sí	Tiene usuario, una @ y un punto en el dominio.
jose.perez@upn.edu.pe	Sí	Puede haber puntos en el usuario y en el dominio.
ana.gmail.com	No	Falta la @.
ana@gmail	No	El dominio no tiene punto.
ana@@gmail.com	No	Tiene dos @.
ana @gmail.com	No	Tiene un espacio.

Es una revisión básica de formato: no comprueba que el correo exista de verdad.

5. Varios pedidos seguidos y cierre de caja

Antes el programa atendía **un solo pedido** y se cerraba, así que el control de stock casi no se notaba. Ahora, al terminar cada boleta pregunta *"¿Registrar otro pedido? (S/N)"*. El stock que se vendió en un pedido ya no está disponible para el siguiente. Cada boleta tiene número (0001, 0002…). Al responder N se muestra el cierre de caja:

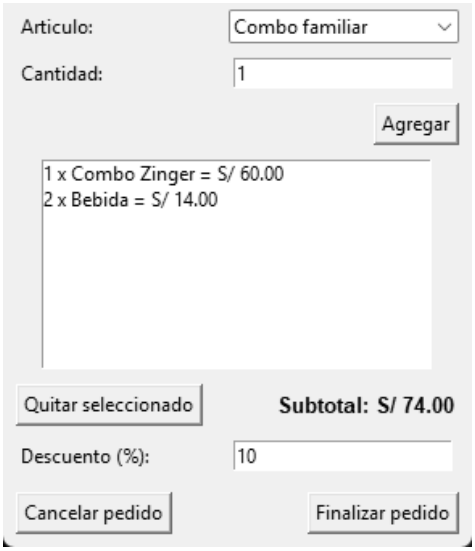
```
========================================
=== CIERRE DE CAJA - BROASTER GO ===
========================================
Pedidos confirmados: 2
Pedidos rechazados:  1
Total vendido:       S/ 88.90

Ventas por forma de pago:
 - Efectivo: S/ 76.40
 - Yape: S/ 12.50
 - Plin: S/ 0.00
 - Tarjeta: S/ 0.00

Stock final:
 - Pieza de Pollo Broaster: 9
 - Combo 2 Piezas + Papas: 1
 ...
```

Las salidas también quedaron separadas en funciones con nombres claros, como en la tabla de capas del README: `emitir_boleta()`, `notificar_cliente()`, `enviar_comanda()`, `mostrar_incidencias()` y `mostrar_cierre_caja()`. Si un pedido se rechaza, no se envía comanda a cocina ni notificación al cliente.

6. Ventana de mostrador (Tkinter)



Qué cambió:

- **Quitar seleccionado:** se elige una línea de la lista y se elimina.
- **Subtotal en vivo** debajo de la lista, se actualiza al agregar o quitar.
- **Enter** en "Cantidad" agrega el artículo, sin usar el ratón.
- El descuento debe estar **entre 0 y 100**; si no, aparece un mensaje de error.
- **Finalizar pedido** muestra el detalle y deja la ventana limpia para el siguiente cliente.
- **Cancelar pedido** borra todo, después de pedir confirmación.

Por dentro: la clase VentanaPedido

El código pasó de variables sueltas a una clase `VentanaPedido`, el mismo nombre que figura en la sección 6.2 del README. Cada botón llama a un método de la clase:

```
class VentanaPedido:
    def agregar(self, evento=None): ...   # boton "Agregar" o tecla Enter
    def quitar(self): ...                 # boton "Quitar seleccionado"
    def finalizar(self): ...              # muestra el total y limpia la ventana
    def cancelar(self): ...               # pregunta y borra el pedido
    def limpiar(self): ...                # deja todo listo para otro cliente
```

El parámetro `evento=None` existe porque tkinter envía un "evento" cuando se presiona Enter; cuando se usa el botón, no envía nada y queda en None.

7. Orden del repositorio

.gitignore y __pycache__. La carpeta *__pycache__* la crea Python automáticamente y no debe subirse a GitHub. Se borró del repositorio y se agregó un archivo `.gitignore` para que no vuelva a subirse.

if __name__ == "__main__". Antes los programas arrancaban apenas se importaban, así que no se podían probar. Ahora el programa principal solo se ejecuta cuando corres el archivo directamente:

```
def main():
    ...                       # el programa de siempre

if __name__ == "__main__":    # True solo con: python broaster_go_pedidos.py
    main()                    # al hacer "import", no se ejecuta
```

8. Pruebas automáticas

Una prueba automática es un pequeño programa que usa tu código con datos conocidos y revisa que el resultado sea el esperado. Se usa `unittest`, que ya viene con Python. Así, cada vez que cambies algo, sabrás en un segundo si rompiste alguna regla.

```
> python -m unittest -v
test_r4_delivery_debajo_del_minimo_se_rechaza_y_devuelve_stock ... ok
test_r5_delivery_suma_la_tarifa_de_la_zona ... ok
...
----------------------------------------------------------------------
Ran 24 tests in 0.001s

OK
```

Archivo	Grupo	Pruebas	Qué revisa
test_calculos.py	Cálculos	5	IGV, descuento antes del IGV, porcentajes fuera de rango, formato en soles.
test_pedidos.py	Cliente	4	Correos válidos e inválidos, cliente con datos vacíos.
test_pedidos.py	Procesar pedido	14	Una o más pruebas para cada regla R1 a R8, incluido que el stock se devuelva.
test_pedidos.py	Cierre de caja	1	Suma solo los pedidos confirmados y separa por forma de pago.

Cómo se ve una prueba

```
def test_r5_delivery_suma_la_tarifa_de_la_zona(self):
    resultado = SistemaPedidos.procesar_pedido(
        cliente_de_prueba(), [(self.combo, 2)], zona="c"   # 36.00 + 8.00
    )
    self.assertEqual(resultado["delivery"], 8.0)
    self.assertEqual(resultado["total"], 44.0)
```

Para agregar una prueba nueva: copia una existente, cambia el nombre (debe empezar con `test_`), los datos y el valor esperado en `assertEqual`.

Ideas para seguir mejorando

- **Guardar los datos con SQLite** (`sqlite3` viene con Python): hoy el stock vuelve a su valor inicial al cerrar el programa. Es la capa “BaseDatos” que ya está diseñada en el README.
- **Guardar cada boleta en un archivo .txt** con `open()`, para tener un respaldo del día.
- **Conectar la ventana con el sistema POO**, para que también controle stock y delivery.
- **GitHub Actions**: que GitHub ejecute las pruebas automáticamente en cada cambio.